Universidad Tecnologica Nacional

[Facultad Regional .....]

KICKET

Sistema de gestion de entradas para espectaculos deportivos

Trabajo Practico Integrador – Entrega Final

Institucion	Universidad Tecnologica Nacional – **[Facultad Regional .....]**
Asignatura	**[Completar asignatura]**
Comision	**[Completar comision]**
Docentes	**[Completar docente/s a cargo]**
Grupo	**[Completar numero / nombre de grupo]**
Ciclo lectivo	2025
Fecha de entrega	**[Completar fecha]**

Integrantes

N°	Apellido	Nombre	Legajo
1	**[Apellido]**	**[Nombre]**	**[Legajo]**
2	**[Apellido]**	**[Nombre]**	**[Legajo]**
3	**[Apellido]**	**[Nombre]**	**[Legajo]**
4	**[Apellido]**	**[Nombre]**	**[Legajo]**
5	**[Apellido]**	**[Nombre]**	**[Legajo]**

Repositorio de codigo fuente

URL del repositorio	https://github.com/G-rm4n/Kicket
Rama de la entrega	**[Completar rama / tag de la entrega final]**

El repositorio contiene la solucion completa (API REST, interfaz de escritorio e interfaz web) junto con los datos de prueba necesarios para ejecutar y validar el sistema. Las carpetas *bin*, *obj*, *.vs* y los archivos *.suo* / *.user* se encuentran excluidos mediante el archivo *.gitignore*, por lo que no forman

parte del paquete comprimido entregado.

1. Descripcion general del sistema

Kicket es un sistema de gestion destinado a la administracion de la informacion necesaria para la venta de entradas de espectaculos deportivos. El sistema centraliza la informacion de los clubes participantes, de los estadios donde se disputan los encuentros y de los usuarios habilitados para operar la plataforma, aplicando distintos niveles de acceso segun el perfil asignado a cada usuario.

La solucion se compone de tres aplicaciones que comparten una unica capa de dominio y una unica capa de servicios: una API REST que concentra la logica de negocio y la emision de tokens de seguridad, una aplicacion de escritorio desarrollada en Windows Forms y una aplicacion web desarrollada en ASP.NET Core MVC. Tanto la interfaz de escritorio como la interfaz web consumen exclusivamente los recursos publicados por la API, de manera que la logica de negocio no se encuentra duplicada en los clientes.

Ambas interfaces implementan la totalidad de las operaciones de alta, baja, modificacion y consulta sobre las tres entidades del dominio, y ofrecen dos reportes gerenciales, uno de los cuales incluye una representacion grafica de los datos.

1.1. Requerimientos funcionales

Codigo	Requerimiento	Descripcion
RF-01	Autenticacion de usuarios	El sistema debe permitir que un usuario registrado inicie sesion indicando su correo electronico y su contrasena, y debe emitir un token de acceso valido por tiempo limitado.
RF-02	Cierre de sesion	El sistema debe permitir finalizar la sesion activa, descartando el token y las credenciales almacenadas en el cliente.
RF-03	Autorizacion por perfil	El sistema debe habilitar u ocultar las funcionalidades segun el perfil del usuario autenticado (Admin, Vendedor o User).
RF-04	Gestion de clubes	El sistema debe permitir listar, consultar en detalle, registrar, modificar y eliminar clubes.
RF-05	Gestion de estadios	El sistema debe permitir listar, consultar en detalle, registrar, modificar y eliminar estadios, incluyendo su domicilio y ciudad.
RF-06	Gestion de usuarios	El sistema debe permitir listar, consultar en detalle, registrar, modificar y eliminar usuarios, incluyendo la asignacion del perfil.
RF-07	Validacion de datos obligatorios	El sistema debe rechazar las operaciones de alta y modificacion cuando los campos obligatorios de cada entidad se encuentren vacios.
RF-08	Reporte de usuarios por perfil	El sistema debe emitir un reporte que indique la cantidad de usuarios y la edad promedio por cada perfil, acompanado de un grafico de barras.
RF-09	Reporte de estadios por ciudad	El sistema debe emitir un reporte que indique la cantidad de estadios por ciudad, el detalle de los estadios y su participacion porcentual.

RF-10	Disponibilidad en dos interfaces	La totalidad de las funcionalidades anteriores debe estar disponible tanto en la interfaz de escritorio como en la interfaz web.

1.2. Requerimientos no funcionales

Codigo	Requerimiento	Descripcion
RNF-01	Arquitectura en capas	La solucion debe respetar la separacion entre la capa de presentacion, la capa de servicios y la capa de dominio, sin que las interfaces accedan directamente a la logica de negocio.
RNF-02	Seguridad basada en tokens	Toda operacion distinta del inicio de sesion debe requerir un token JWT valido, firmado con algoritmo HMAC SHA-256.
RNF-03	Expiracion de credenciales	El token emitido debe expirar automaticamente a los 120 minutos, valor configurable desde el archivo de configuracion de la API.
RNF-04	Usabilidad	Las interfaces deben presentar una identidad visual homogenea, mensajes de error claros y confirmacion explicita ante operaciones de baja.
RNF-05	Mantenibilidad	El acceso a datos debe resolverse detras de interfaces de servicio, de modo que el reemplazo del mecanismo de persistencia no impacte en las interfaces de usuario.
RNF-06	Interoperabilidad	La API debe exponer sus recursos en formato JSON sobre HTTP y documentarse mediante OpenAPI / Swagger.
RNF-07	Portabilidad del cliente web	La interfaz web debe adaptarse a distintos tamanos de pantalla mediante hojas de estilo propias.
RNF-08	Datos de prueba	El sistema debe inicializarse con un juego de datos representativo que permita validar la totalidad de las funcionalidades sin carga manual previa.

2. Capturas de pantalla

Las capturas que se presentan a continuacion corresponden a la ejecucion del sistema con el juego de datos de prueba incluido en el repositorio.

2.1. Interfaces de escritorio

[Insertar captura: inicio de sesion de la aplicacion de escritorio]

Figura 1. Formulario de inicio de sesion (Windows Forms).

[Insertar captura: panel principal con el menu segun el perfil]

Figura 2. Panel principal con el menu habilitado segun el perfil del usuario.

[Insertar captura: listado de clubes]

Figura 3. Administracion de clubes.

[Insertar captura: alta / edicion de club]

Figura 4. Formulario de alta y modificacion de clubes.

[Insertar captura: listado de estadios]

Figura 5. Administracion de estadios.

[Insertar captura: listado de usuarios]

Figura 6. Administracion de usuarios y asignacion de perfiles.

2.2. Interfaces web

[Insertar captura: inicio de sesion de la aplicacion web]

Figura 7. Formulario de inicio de sesion (ASP.NET Core MVC).

[Insertar captura: pagina de inicio]

Figura 8. Pagina de inicio con el perfil del usuario autenticado.

[Insertar captura: listado de clubes]

Figura 9. Listado de clubes.

[Insertar captura: formulario de alta de estadio]

Figura 10. Alta de estadios.

[Insertar captura: confirmacion de baja]

Figura 11. Confirmacion de baja de un registro.

[Insertar captura: mensaje de acceso denegado]

Figura 12. Restriccion de acceso para un perfil sin permisos.

2.3. Reportes

[Insertar captura: reporte de usuarios por perfil en escritorio, con el grafico de barras]

Figura 13. Reporte de usuarios por perfil – interfaz de escritorio.

[Insertar captura: reporte de estadios por ciudad en escritorio]

Figura 14. Reporte de estadios por ciudad – interfaz de escritorio.

[Insertar captura: reporte de usuarios por perfil en la web, con el grafico Chart.js]

Figura 15. Reporte de usuarios por perfil – interfaz web.

[Insertar captura: reporte de estadios por ciudad en la web]

Figura 16. Reporte de estadios por ciudad – interfaz web.

3. Modelo de datos

El modelo logico de datos del sistema esta conformado por tres entidades independientes entre si en su estado actual: Usuario, Club y Estadio. La persistencia se resuelve mediante repositorios en memoria alojados en la capa de servicios, registrados como instancias unicas durante la ejecucion de la API, por lo que el modelo que se describe a continuacion representa el esquema relacional equivalente que adoptaria la solucion al incorporar un motor de base de datos.

3.1. Tablas

Usuario

Campo	Tipo	Clave	Descripcion
IdUsuario	int	PK	Identificador unico del usuario.
DNI	varchar(10)	UK	Documento nacional de identidad.
Nombre	varchar(60)		Nombre de pila.
Apellido	varchar(60)		Apellido.
Email	varchar(120)	UK	Correo electronico, utilizado como identificador de inicio de sesion.
Pass	varchar(40)		Contrasena de acceso.
FechaNacimiento	date		Fecha de nacimiento, base del calculo de edad en los reportes.
Rol	varchar(20)		Perfil asignado: Admin, Vendedor o User.

Club

Campo	Tipo	Clave	Descripcion
IdClub	int	PK	Identificador unico del club.
Nombre	varchar(80)	UK	Denominacion del club.
Logo_Url	varchar(200)		Direccion del recurso grafico institucional.

Estadio

Campo	Tipo	Clave	Descripcion
IdEstadio	int	PK	Identificador unico del estadio.
Nombre	varchar(80)		Denominacion del estadio.
Calle	varchar(80)		Calle del domicilio.
Nro	int		Numeracion del domicilio.
Ciudad	varchar(80)		Ciudad, base de la agrupacion del reporte geografico.

3.2. Diagrama de tablas y relaciones

[Insertar diagrama de entidad-relacion exportado desde la herramienta utilizada]

Figura 17. Diagrama de tablas y relaciones.

```
+---------------------------+      +-----------------------+
+---------------------------+
|          USUARIO          |      |          CLUB         |      |          ESTADIO
|
+---------------------------+      +-----------------------+
+---------------------------+
| PK IdUsuario    int       |      | PK IdClub      int    |      | PK IdEstadio    int
|
|    DNI          varchar   |      |    Nombre      varchar |      |    Nombre       varchar
|
|    Nombre       varchar   |      |    Logo_Url    varchar |      |    Calle        varchar
|
|    Apellido     varchar   |      +-----------------------+      |    Nro          int
|
|    Email        varchar   |                                     |    Ciudad       varchar
|
|    Pass         varchar   |
+---------------------------+
|    FechaNacim.  date      |
|    Rol          varchar   |
+---------------------------+
```

El campo Rol de la entidad Usuario determina el perfil de seguridad aplicado por la API y constituye la dimension de agrupacion del primer reporte; el campo Ciudad de la entidad Estadio constituye la dimension de agrupacion del segundo reporte.

4. Modelo de clases

El modelo de clases se distribuye en tres proyectos: el proyecto de dominio, que contiene las entidades y los objetos de transferencia de datos; el proyecto de servicios, que contiene las interfaces y sus implementaciones; y los proyectos de presentacion, que contienen los controladores, los formularios y los clientes de la API.

4.1. Capa de dominio

Clase	Responsabilidad
Usuario	Entidad que representa a un usuario del sistema junto con su perfil de seguridad.
Club	Entidad que representa a un club participante.
Estadio	Entidad que representa a un estadio y su ubicacion.
LoginRequest	Objeto de transferencia con las credenciales enviadas al iniciar sesion.
LoginResponse	Objeto de transferencia con el token emitido, los datos del usuario y la fecha de expiracion.
UsuariosPorRol	Objeto de transferencia con el resultado del reporte de usuarios por perfil.
EstadiosPorCiudad	Objeto de transferencia con el resultado del reporte de estadios por ciudad.

4.2. Capa de servicios

Interfaz	Implementacion	Responsabilidad
IUsuarioService	UsuarioService	Operaciones de alta, baja, modificacion, consulta y busqueda por correo electronico de usuarios.
IClubService	ClubService	Operaciones de alta, baja, modificacion y consulta de clubes.
IEstadioService	EstadioService	Operaciones de alta, baja, modificacion y consulta de estadios.
IAuthService	AuthService	Validacion de credenciales y generacion del token JWT firmado.
IReporteService	ReporteService	Agrupacion y calculo de los indicadores de ambos reportes.

4.3. Capa de presentacion

Componente	Proyecto	Responsabilidad
AuthController	Kicket.API	Publica el inicio de sesion y la consulta del perfil autenticado.
UsuarioController, ClubController, EstadioController	Kicket.API	Publican los recursos REST de cada entidad con sus reglas de autorizacion.
ReporteController	Kicket.API	Publica los recursos de ambos reportes.
ApiClient, Sesion	Kicket.Escritorio	Consumo de la API y conservacion del token y del perfil durante la sesion.
LoginForm, PrincipalForm	Kicket.Escritorio	Inicio de sesion y navegacion principal

		con menu condicionado por perfil.
ClubesForm, EstadiosForm, UsuariosForm y sus formularios de edicion	Kicket.Escritorio	Operaciones de alta, baja, modificacion y consulta de cada entidad.
ReporteUsuariosForm, ReporteEstadiosForm	Kicket.Escritorio	Emision de los reportes; el primero incorpora el grafico de barras dibujado con GDI+.
ApiClient	Kicket.Web	Consumo de la API adjuntando el token conservado en la cookie de autenticacion.
CuentaController, HomeController	Kicket.Web	Inicio y cierre de sesion, pagina principal y pagina de acceso denegado.
ClubesController, EstadiosController, UsuariosController	Kicket.Web	Operaciones de alta, baja, modificacion y consulta de cada entidad.
ReportesController	Kicket.Web	Emision de los reportes; el primero incorpora el grafico generado con Chart.js.

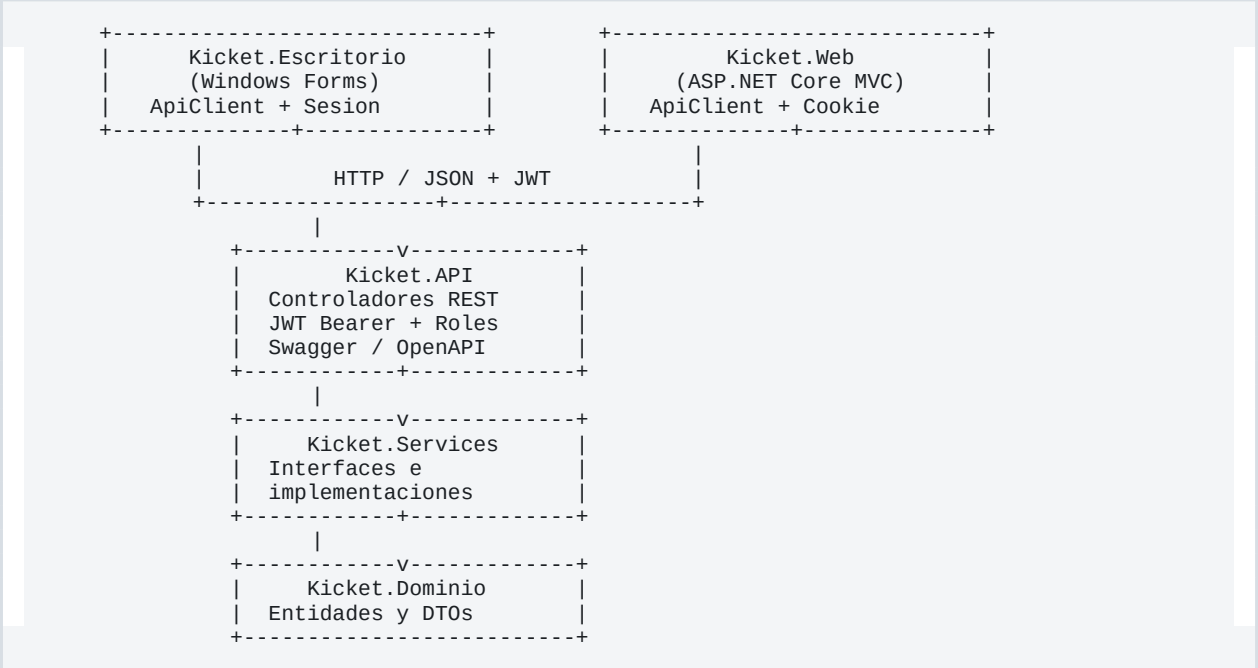
[Insertar diagrama de clases exportado desde la herramienta utilizada]

Figura 18. Diagrama de clases de la solucion.

5. Arquitectura implementada

La solucion adopta una arquitectura en capas con un servicio central de aplicacion. La capa de dominio no posee dependencias y define las entidades y los objetos de transferencia que comparten todos los proyectos. La capa de servicios depende unicamente del dominio y concentra la logica de negocio, la validacion de credenciales, la generacion de los tokens y el calculo de los reportes, exponiendo cada responsabilidad detras de una interfaz. La capa de presentacion se compone de tres proyectos: la API REST, que publica la logica de servicios como recursos HTTP y resuelve la autenticacion y la autorizacion; la aplicacion de escritorio; y la aplicacion web.

Las dos interfaces de usuario no referencian la capa de servicios: ambas consumen la API mediante un cliente HTTP propio que adjunta el token de acceso en el encabezado de autorizacion. De esta manera, la logica de negocio permanece en un unico punto y cualquier modificacion de las reglas impacta simultaneamente en ambas interfaces. La inyeccion de dependencias se resuelve en el arranque de la API mediante el contenedor nativo de ASP.NET Core, registrando cada interfaz de servicio contra su implementacion.



5.1. Seguridad implementada

La autenticacion se resuelve mediante tokens JWT. El recurso *POST api/Auth/login* valida las credenciales recibidas y, si son correctas, emite un token firmado con el algoritmo HMAC SHA-256 que incluye el identificador, el nombre, el correo electronico y el perfil del usuario, junto con una fecha de expiracion de 120 minutos. La clave de firma, el emisor, la audiencia y el tiempo de expiracion se configuran en la seccion *Jwt* del archivo appsettings.json de la API.

Todos los recursos restantes exigen un token valido. La autorizacion se aplica por perfil mediante

atributos declarativos sobre los controladores y las acciones, segun el siguiente esquema:

Recurso	Operacion	Perfiles habilitados
api/Auth/login	Alta de sesion	Anonimo
api/Club, api/Estadio	Consulta	Admin, Vendedor, User
api/Club, api/Estadio	Alta y modificacion	Admin, Vendedor
api/Club, api/Estadio	Baja	Admin
api/Usuario	Todas	Admin
api/Reporte	Consulta	Admin, Vendedor

Las interfaces de usuario replican estas reglas en la presentacion: la aplicacion de escritorio oculta o deshabilita los menus y los botones no permitidos, y la aplicacion web condiciona los enlaces de navegacion y protege cada accion con atributos de autorizacion, derivando al usuario a una pagina de acceso denegado cuando corresponde.

6. Reportes

6.1. Reporte de usuarios por perfil

Agrupa la totalidad de los usuarios registrados por el perfil asignado e informa, para cada uno, la cantidad de usuarios, la edad promedio calculada a partir de la fecha de nacimiento y la participacion porcentual sobre el total. El reporte se acompana de un grafico de barras que permite comparar visualmente la distribucion de los perfiles. En la interfaz de escritorio el grafico se dibuja con GDI+ sobre un panel de la aplicacion; en la interfaz web se genera con la libreria Chart.js.

6.2. Reporte de estadios por ciudad

Agrupa los estadios registrados por ciudad e informa la cantidad de estadios, el detalle de los nombres de cada uno y la participacion porcentual sobre el total. Permite evaluar la cobertura geografica de la plataforma. Se presenta en formato tabular con un encabezado de indicadores resumen en ambas interfaces.

7. Tecnologias utilizadas

Tecnologia	Version	Utilizacion
Microsoft Visual Studio 2022	17.14	Entorno de desarrollo integrado de la solucion.
.NET	8.0	Plataforma de ejecucion de los cinco proyectos de la solucion.
C#	12	Lenguaje de programacion.
ASP.NET Core Web API	8.0	Publicacion de los recursos REST de la capa de servicios.
ASP.NET Core MVC	8.0	Interfaz web con vistas Razor.
Windows Forms	.NET 8	Interfaz de escritorio.
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.10	Validacion de los tokens recibidos por la API.
System.IdentityModel.Tokens.Jwt	8.0.2	Generacion y firma de los tokens JWT.
Microsoft.AspNetCore.Authentication.Cookies	8.0	Conservacion de la sesion y del token en la interfaz web.
Swashbuckle.AspNetCore	10.2.3	Documentacion interactiva de la API mediante Swagger / OpenAPI.
System.Net.Http.Json	8.0	Serializacion y deserializacion de los mensajes intercambiados con la API.
Chart.js	4.4.1	Motor de graficos del reporte de la interfaz web.
GDI+ (System.Drawing)	.NET 8	Motor de graficos del reporte de la

		interfaz de escritorio.
Git y GitHub	–	Control de versiones y publicacion del codigo fuente.

8. Datos de prueba

La solucion se inicializa con un juego de datos representativo que permite validar la totalidad de las funcionalidades sin necesidad de carga manual previa. Los datos se cargan al instanciarse los servicios de la API.

8.1. Usuarios y credenciales de acceso

ID	Apellido y nombre	Email	Contrasena	Perfil
1	Petazzi, Juan Cruz	admin@kicket.com.ar	Admin123	Admin
2	Suarez, Marina	marina.suarez@kicket.com.ar	Admin123	Admin
3	Fernandez, Lucas	lucas.fernandez@gmail.com	User123	User
4	Ortega, Camila	camila.ortega@gmail.com	User123	User
5	Gimenez, Matias	matias.gimenez@hotmail.com	User123	User
6	Ledesma, Sofia	sofia.ledesma@gmail.com	User123	User
7	Alvarez, Tomas	tomas.alvarez@outlook.com	User123	User
8	Benitez, Rodrigo	rodrigo.benitez@kicket.com.ar	Vend123	Vendedor
9	Rios, Valentina	valentina.rios@kicket.com.ar	Vend123	Vendedor

8.2. Clubes

ID	Nombre	Logo
1	Rosario Central	https://kicket.com.ar/logos/rosario-central.png
2	Newell's Old Boys	https://kicket.com.ar/logos/newells.png
3	River Plate	https://kicket.com.ar/logos/river-plate.png
4	Boca Juniors	https://kicket.com.ar/logos/boca-juniors.png
5	Racing Club	https://kicket.com.ar/logos/racing-club.png
6	Independiente	https://kicket.com.ar/logos/independiente.png
7	San Lorenzo	https://kicket.com.ar/logos/san-lorenzo.png
8	Talleres	https://kicket.com.ar/logos/talleres.png

8.3. Estadios

ID	Nombre	Direccion	Ciudad
1	Gigante de Arroyito	Avenida Genova 640	Rosario
2	Marcelo Bielsa	Pasaje Juan B. Justo 3550	Rosario
3	Mas Monumental	Avenida Figueroa Alcorta 7597	Buenos Aires
4	La Bombonera	Brandsen 805	Buenos Aires
5	Pedro Bidegain	Avenida Perito Moreno 1750	Buenos Aires

6	Presidente Peron	Mozart 2500	Avellaneda
7	Libertadores de America	Bochini 751	Avellaneda
8	Mario Alberto Kempes	Avenida Ramon Carcano 1550	Cordoba

9. Instrucciones de ejecucion

Para ejecutar y validar la solucion se deben seguir los siguientes pasos:

Paso	Accion
1	Abrir el archivo Kiketpropio.sln con Microsoft Visual Studio 2022 y restaurar los paquetes NuGet de la solucion.
2	Iniciar el proyecto Kicket.API con el perfil http, disponible en http://localhost:5013. La documentacion interactiva se encuentra en http://localhost:5013/swagger.
3	Para validar la interfaz web, iniciar el proyecto Kicket.Web, disponible en http://localhost:5120, e ingresar con alguna de las credenciales de prueba.
4	Para validar la interfaz de escritorio, iniciar el proyecto Kicket.Escritorio e ingresar con alguna de las credenciales de prueba.
5	Ambas interfaces requieren que la API se encuentre en ejecucion. La direccion de la API se configura en la clave ApiUrl del archivo appsettings.json del proyecto web y en la constante UrlBase del cliente de la aplicacion de escritorio.

Dado que la persistencia se resuelve en memoria, los datos vuelven a su estado inicial cada vez que se reinicia la API, lo que facilita la repeticion de las pruebas.